

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la computación



Carrera de Ingeniería en Software

Tarea 1.2: Desarrollo aplicaciones móviles con controles básicos en Flutter

Asignatura: Desarrollo de Aplicaciones Móviles

Integrantes:

- Pilaguano Chisaguano David Alexander
- Asmal Haro Kevin David
- Villarroel Barreno Justin Joshua
- Domínguez Aguilera Pablo Alejandro

NRC: 30737

Fecha: 12 de mayo de 2026

Índice

Resumen	3
1. Introducción	4
2. Marco Técnico	5
2.1. Patrón MVC	5
2.2. Atomic Design	5
2.3. Controles de Flutter	6
3. Desarrollo de la Aplicación 1: Pagos de Servicios Básicos	6
3.1. Descripción del Problema	6
3.2. Lógica de Negocio	6
3.3. Componentes de Interfaz	7
3.4. Código del programa	7
3.5. Código en la carpeta controlador	7
3.6. Código en la carpeta model	8
3.7. Código en la carpeta view	8
3.8. Código en la carpeta widgets	10
3.9. Código en la carpeta atoms	10
3.10. Código en la carpeta molecules	11
3.11. Código en el archivo main.dart	11
3.12. Programa Funcional	12
4. Desarrollo de la Aplicación 2: Factura de Pago a Choferes	17
4.1. Descripción del Problema	17
4.2. Cálculos Implementados	17
4.3. Controles Específicos	17
4.4. Código del programa	18
4.5. Código en la carpeta controlador	18
4.6. Código en la carpeta model	18
4.7. Código en la carpeta view	19

4.8. Código en la carpeta widgets	20
4.9. Código en la carpeta atoms	20
4.10. Código en la carpeta molecules	21
4.11. Código en el archivo main.dart	22
4.12. Programa Funcional	22
5. Conclusiones	25

Resumen

Este trabajo documenta el desarrollo de dos aplicaciones móviles independientes creadas con el framework Flutter, cuyo objetivo principal es la implementación práctica del patrón de arquitectura MVC (Modelo-Vista-Controlador) y la metodología de diseño *Atomic Design*.

La primera aplicación está orientada a la gestión de pagos de servicios básicos (agua, luz, internet, entre otros), permitiendo el registro de datos del cliente, el cálculo de subtotales, la aplicación de tarifas y la generación de resúmenes de pago. La segunda aplicación consiste en un sistema de facturación para choferes, diseñado para calcular la nómina semanal basándose en horas trabajadas y sueldos por hora, incluyendo reportes específicos como el total general de la empresa y estadísticas diarias.

Ambos proyectos integran controles fundamentales de Flutter (*TextFields*, *CheckBox*, *RadioButtons*), una estructura de navegación basada en rutas (*push*, *pop*, *push-Named*) y una organización de archivos estricta para garantizar una interfaz clara, funcional y profesional.

Palabras clave: Flutter, MVC, Atomic Design, Aplicaciones móviles, Navegación

1. Introducción

En el actual ecosistema del desarrollo de software móvil, la eficiencia y la escalabilidad dependen directamente de la adopción de arquitecturas organizadas y metodologías de diseño modulares. El presente proyecto detalla el diseño y la implementación de dos soluciones móviles desarrolladas con el framework Flutter, orientadas a resolver problemas cotidianos de gestión de datos y cálculos financieros mediante el uso de controles básicos y estructuras profesionales.

El desarrollo se fundamenta en dos pilares arquitectónicos esenciales: el patrón MVC (*Modelo-Vista-Controlador*), que permite una separación clara entre la lógica de negocio y la interfaz de usuario, y la metodología *Atomic Design*, la cual organiza los componentes visuales en átomos, moléculas y organismos para fomentar la reutilización de código.

Para demostrar la versatilidad de estas prácticas, se han diseñado dos aplicaciones con propósitos distintos pero bajo un mismo estándar técnico:

- **Gestión de Pagos de Servicios:** Una herramienta enfocada en el registro y cálculo de consumos para servicios esenciales como agua, energía eléctrica e internet, integrando reglas de negocio para la aplicación de tarifas y descuentos.
- **Sistema de Nómina para Choferes:** Una aplicación administrativa destinada a computar las jornadas laborales semanales, sueldos y estadísticas operativas de una flota de transporte.

A través de la implementación de rutas de navegación y una estructura de carpetas rigurosa (*models, controllers, views, widgets*), este trabajo busca no solo cumplir con una funcionalidad técnica, sino también ofrecer una interfaz clara, ordenada y profesional que cumpla con los estándares modernos de usabilidad.

2. Marco Técnico

En el desarrollo de las aplicaciones móviles se utilizaron tecnologías y metodologías modernas que permiten una mejor organización del código, reutilización de componentes y facilidad de mantenimiento. Entre los principales enfoques implementados se encuentran el patrón MVC, la metodología *Atomic Design* y el uso de controles fundamentales de Flutter.

2.1. Patrón MVC

El patrón arquitectónico MVC (*Modelo-Vista-Controlador*) fue utilizado para separar las responsabilidades del sistema y mejorar la organización del proyecto.

- **Modelo (Model):** Encargado de representar y gestionar los datos utilizados por las aplicaciones, incluyendo variables, estructuras y entidades relacionadas con servicios básicos y choferes.
- **Vista (View):** Corresponde a la interfaz gráfica presentada al usuario. Aquí se implementaron formularios, botones, etiquetas y componentes visuales desarrollados en Flutter.
- **Controlador (Controller):** Responsable de procesar la lógica del sistema, realizar cálculos, validar datos y coordinar la interacción entre el modelo y la vista.

2.2. Atomic Design

La metodología *Atomic Design* permitió estructurar los componentes visuales de manera jerárquica dentro de la carpeta *widgets*, facilitando la reutilización y modularidad de la interfaz.

- **Átomos (Atoms):** Componentes básicos como botones, etiquetas de texto y campos de entrada.
- **Moléculas (Molecules):** Conjunto de átomos combinados para formar elementos funcionales, como formularios de registro o tarjetas informativas.
- **Organismos (Organisms):** Secciones completas de la interfaz conformadas por múltiples moléculas y átomos, utilizadas para construir pantallas completas.

2.3. Controles de Flutter

Las aplicaciones integraron diferentes controles básicos proporcionados por Flutter para mejorar la interacción con el usuario.

- **TextFields:** Utilizados para el ingreso de datos como nombres, horas trabajadas, consumos y valores monetarios.
- **Labels o Text:** Empleados para mostrar resultados, subtotales, mensajes y reportes generados por las aplicaciones.
- **Buttons:** Implementados para ejecutar acciones como calcular valores, limpiar formularios y navegar entre pantallas.

3. Desarrollo de la Aplicación 1: Pagos de Servicios Básicos

3.1. Descripción del Problema

La primera aplicación desarrollada tiene como objetivo gestionar el registro y cálculo de pagos relacionados con servicios básicos como agua potable, energía eléctrica, internet y televisión por cable. El sistema permite ingresar información del cliente y calcular automáticamente los valores correspondientes según el servicio seleccionado.

3.2. Lógica de Negocio

La aplicación implementa reglas de negocio para calcular el subtotal del servicio seleccionado, aplicar tarifas específicas y considerar descuentos o recargos dependiendo del tipo de servicio contratado y de opciones adicionales seleccionadas por el usuario.

Los cálculos son procesados en el controlador, permitiendo mantener separada la lógica de la interfaz gráfica y facilitando futuras modificaciones del sistema.

3.3. Componentes de Interfaz

Para mejorar la experiencia del usuario se utilizaron diversos controles visuales de Flutter:

- **RadioButton:** Utilizados para seleccionar el tipo de servicio básico a cancelar.
- **CheckBox:** Empleados para activar servicios adicionales o beneficios complementarios.
- **TextFields:** Implementados para el ingreso de datos del cliente y valores de consumo.
- **Buttons:** Utilizados para ejecutar operaciones de cálculo y limpieza de datos.

3.4. Código del programa

■ 3.5. Código en la carpeta controlador

Ruta: lib/controller/paymentcontroller.dart

Listing 1: Código del controlador

```
1 import 'package:pagos_servicios_basicos/model/payment_model.dart';
2
3 class PaymentController {
4   // Tarifas por unidad
5   final Map<String, double> rates = {
6     'Agua': 0.75,
7     'Luz': 0.12,
8     'Internet': 30.0,
9     'TV/Cable': 25.0,
10  };
11
12  double calculateSubtotal(PaymentModel data) {
13    double rate = rates[data.serviceType] ?? 0.0;
14
15    // Internet y TV suelen ser tarifa plana
16    if (data.serviceType == 'Internet' ||
17        data.serviceType == 'TV/Cable') {
18      return rate;
19    }
20
21    return data.consumption * rate;
22  }
23
24  double calculateTotal(PaymentModel data) {
25    double subtotal = calculateSubtotal(data);
26    double total = subtotal;
27
28    // Lógica automática basada en la edad
29    if (data.age >= 65) {
30      data.isSeniorDiscount = true;
31      total -= subtotal * 0.50;
32    } else {
33      data.isSeniorDiscount = false;
34    }
35
36    if (data.hasLateFee)
37      total += subtotal * 0.10;
38
39    return total;
```



```
40 }  
41 }
```

■ 3.6. Código en la carpeta model

Ruta: lib/model/paymentmodel.dart

Listing 2: Código en la carpeta model

```
1 class PaymentModel {  
2   String clientName;  
3   String serviceType;  
4   double consumption;  
5   bool hasLateFee;  
6   bool isSeniorDiscount;  
7   int age;  
8  
9   PaymentModel({  
10    this.clientName = '',  
11    this.serviceType = 'Agua',  
12    this.consumption = 0.0,  
13    this.hasLateFee = false,  
14    this.isSeniorDiscount = false,  
15    this.age = 0,  
16  });  
17 }
```

■ 3.7. Código en la carpeta view

Ruta: lib/view/paymentformview.dart

Listing 3: Código en la carpeta view

```
1 import 'package:flutter/material.dart';  
2 import 'package:pagos_servicios_basicos/controller/payment_controller.dart';  
3 import 'package:pagos_servicios_basicos/model/payment_model.dart';  
4  
5 import '../widgets/atoms/custom_button.dart';  
6 import '../widgets/atoms/custom_text.dart';  
7 import '../widgets/molecules/labeled_input.dart';  
8  
9 class PaymentFormView extends StatefulWidget {  
10   const PaymentFormView({super.key});  
11  
12   @override  
13   State<PaymentFormView> createState() => _PaymentFormViewState();  
14 }  
15  
16 class _PaymentFormViewState extends State<PaymentFormView> {  
17   // Controladores para los campos de texto  
18   final _nameController = TextEditingController();  
19   final _ageController = TextEditingController();  
20   final _valController = TextEditingController();  
21  
22   // Instancia del controlador de lógica  
23   final _controller = PaymentController();  
24  
25   // Instancia del modelo para manejar el estado  
26   PaymentModel _data = PaymentModel();  
27  
28   void _calculate() {  
29     setState(() {  
30       _data.clientName = _nameController.text;  
31  
32       // CORRECCION AQUÍ : Usar ?? 0 para asegurar que siempre sea un int  
33       _data.age = int.tryParse(_ageController.text) ?? 0;  
34  
35       _data.consumption = double.tryParse(_valController.text) ?? 0.0;  
36     });  
37  
38     double total = _controller.calculateTotal(_data);  
39   }
```

```

40     Navigator.pushNamed(
41         context,
42         '/summary',
43         arguments: {'model': _data, 'total': total},
44     );
45 }
46
47 void _clear() {
48     setState(() {
49         _nameController.clear();
50         _ageController.clear();
51         _valController.clear();
52         _data = PaymentModel(); // Resetear el modelo a valores iniciales
53     });
54 }
55
56 @override
57 Widget build(BuildContext context) {
58     return Scaffold(
59         appBar: AppBar(
60             title: const Text("Registro de Pago de Servicios"),
61             backgroundColor: Colors.blueAccent,
62             foregroundColor: Colors.white,
63         ),
64         body: SingleChildScrollView(
65             padding: const EdgeInsets.all(20),
66             child: Column(
67                 crossAxisAlignment: CrossAxisAlignment.start,
68                 children: [
69                     // --- SECCI N: DATOS DEL CLIENTE ---
70                     const CustomText(
71                         text: "Datos del Cliente",
72                         fontSize: 18,
73                         fontWeight: FontWeight.bold,
74                     ),
75                     const SizedBox(height: 10),
76                     LabeledInput(label: "Nombre completo", controller: _nameController),
77                     LabeledInput(
78                         label: "Edad",
79                         controller: _ageController,
80                         type: TextInputType.number,
81                     ),
82                     const Divider(height: 30),
83                     // --- SECCI N: SERVICIO ---
84                     const CustomText(
85                         text: "Seleccione el Servicio",
86                         fontWeight: FontWeight.bold,
87                     ),
88                     ...['Agua', 'Luz', 'Internet', 'TV/Cable'].map(
89                         (servicio) => RadioListTile(
90                             title: Text(servicio),
91                             value: servicio,
92                             groupValue: _data.serviceType,
93                             onChanged: (v) =>
94                                 setState(() => _data.serviceType = v.toString()),
95                         ),
96                     ),
97                     const SizedBox(height: 10),
98                     LabeledInput(
99                         label: "Consumo (m3/kWh) o Valor Base (\$)",
100                         controller: _valController,
101                         type: TextInputType.number,
102                     ),
103                     const Divider(height: 30),
104                     // --- SECCI N: ADICIONALES ---
105                     const CustomText(
106                         text: "Opciones Adicionales",
107                         fontWeight: FontWeight.bold,
108                     ),
109                     CheckboxListTile(
110                         title: const Text("Recargo por pago fuera de fecha (10%)"),
111                         value: _data.hasLateFee,
112                         onChanged: (v) => setState(() => _data.hasLateFee = v!),
113                         controlAffinity: ListTileControlAffinity.leading,
114                     ),
115                     // Nota informativa sobre la edad
116                     Padding(
117                         padding: const EdgeInsets.symmetric(horizontal: 15, vertical: 5),
118                         child: CustomText(

```

```

125         text: _data.age >= 65
126         ? "      Descuento de 3ra edad aplicado autom ticamente."
127         : "* El descuento de 3ra edad requiere 65 a os o m s.",
128         fontSize: 12,
129         color: _data.age >= 65 ? Colors.green : Colors.grey,
130     ),
131 ),
132
133     const SizedBox(height: 30),
134
135     // --- BOTONES DE ACCI N ---
136     Row(
137       children: [
138         Expanded(
139           child: CustomButton(
140             label: "Calcular Pago",
141             onPressed: _calculate,
142             color: Colors.green,
143           ),
144         ),
145         const SizedBox(width: 15),
146         Expanded(
147           child: CustomButton(
148             label: "Limpiar",
149             onPressed: _clear,
150             color: Colors.redAccent,
151           ),
152         ),
153       ],
154     ),
155   ],
156 ),
157 ),
158 );
159 }
160 }

```

■ 3.8. Código en la carpeta widgets

• 3.9. Código en la carpeta atoms

Ruta: lib/wodgets/atoms/custombutton.dart

Listing 4: Código en la carpeta widgets/atoms

```

1 import 'package:flutter/material.dart';
2
3 class CustomButton extends StatelessWidget {
4   final String label;
5   final VoidCallback onPressed;
6   final Color? color;
7
8   const CustomButton({
9     super.key,
10    required this.label,
11    required this.onPressed,
12    this.color,
13  });
14
15  @override
16  Widget build(BuildContext context) {
17    return ElevatedButton(
18      style: ElevatedButton.styleFrom(
19        backgroundColor: color ?? Colors.blue,
20        foregroundColor: Colors.white,
21      ),
22      onPressed: onPressed,
23      child: Text(label),
24    );
25  }
26 }

```

Ruta: lib/wodgets/atoms/customtext.dart

Listing 5: Código en la carpeta widgets/atoms

```
1 import 'package:flutter/material.dart';
2
3 class CustomText extends StatelessWidget {
4   final String text;
5   final double fontSize;
6   final FontWeight fontWeight;
7   final Color color;
8
9   const CustomText({
10     super.key,
11     required this.text,
12     this.fontSize = 16.0,
13     this.fontWeight = FontWeight.normal,
14     this.color = Colors.black,
15   });
16
17   @override
18   Widget build(BuildContext context) {
19     return Text(
20       text,
21       style: TextStyle(
22         fontSize: fontSize,
23         fontWeight: fontWeight,
24         color: color,
25       ),
26     );
27   }
28 }
```

• 3.10. Código en la carpeta molecules

Ruta: lib/wodgets/atoms/labeledinput.dart

Listing 6: Código en la carpeta widgets/atoms

```
1 import 'package:flutter/material.dart';
2 import '../atoms/custom_text.dart';
3
4 class LabeledInput extends StatelessWidget {
5   final String label;
6   final TextEditingController controller;
7   final TextInputType type;
8
9   const LabeledInput({
10     super.key,
11     required this.label,
12     required this.controller,
13     this.type = TextInputType.text,
14   });
15
16   @override
17   Widget build(BuildContext context) {
18     return Column(
19       crossAxisAlignment: CrossAxisAlignment.start,
20       children: [
21         CustomText(text: label, fontWeight: FontWeight.bold),
22         TextField(controller: controller, keyboardType: type),
23         const SizedBox(height: 15),
24       ],
25     );
26   }
27 }
```

■ 3.11. Código en el archivo main.dart

Ruta: lib/main.dart

Listing 7: Código en la carpeta lib

```
1 import 'package:flutter/material.dart';
2 import 'package:pagos_servicios_basicos/view/payment_form_view.dart';
3 import 'package:pagos_servicios_basicos/view/payment_summary_view.dart';
```

```
4
5 void main() => runApp(const MyApp());
6
7 class MyApp extends StatelessWidget {
8   const MyApp({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return MaterialApp(
13      debugShowCheckedModeBanner: false,
14      title: 'Sistema de Pagos MVC',
15      theme: ThemeData(primarySwatch: Colors.blue),
16      initialRoute: '/',
17      routes: {
18        '/': (context) => const PaymentFormView(),
19        '/summary': (context) => const PaymentSummaryView(),
20      },
21    );
22  }
23 }
```

3.12. Programa Funcional

En esta sección se presentan las capturas de pantalla correspondientes al funcionamiento de la aplicación desarrollada en Flutter. Las imágenes evidencian la implementación de la interfaz gráfica, la interacción con los controles básicos y la correcta ejecución de las funcionalidades programadas.

Se muestran los procesos principales del sistema, incluyendo el ingreso de datos, la selección de opciones mediante controles interactivos, el cálculo automático de resultados y la visualización de reportes generados por la aplicación.

Registro de Pago de Servicios

Datos del Cliente

Nombre completo

David Pilaguano

Edad

18

Seleccione el Servicio

- ☐ Agua
- ☐ Luz
- ☒ Internet
- ☐ TV/Cable

Consumo (m3/kWh) o Valor Base (\$)

52

Opciones Adicionales



Recargo por pago fuera de fecha
(10%)

* El descuento de 3ra edad requiere 65 años o más.

Calcular Pago

Limpiar

← Resumen

Resumen de Pago

Cliente: David Pilaguano

Servicio: Internet

Total Final: \$33.00

Regresar

Registro de Pago de Servicios

Datos del Cliente

Nombre completo

David Pilaguano

Edad

70

Seleccione el Servicio

- ☐ Agua
- ☐ Luz
- ☒ Internet
- ☐ TV/Cable

Consumo (m3/kWh) o Valor Base (\$)

52

Opciones Adicionales



Recargo por pago fuera de fecha
(10%)

✓ Descuento de 3ra edad aplicado automáticamente.

Calcular Pago

Limpiar

← Resumen

Resumen de Pago

Cliente: David Pilaguano

Servicio: Internet

Total Final: \$18.00

Regresar

4. Desarrollo de la Aplicación 2: Factura de Pago a Choferes

4.1. Descripción del Problema

La segunda aplicación corresponde a un sistema de cálculo de nómina semanal para una empresa de transporte. El sistema registra la información de cinco choferes, considerando las horas trabajadas diariamente y el sueldo asignado por hora laboral.

4.2. Cálculos Implementados

La aplicación realiza diversos cálculos automáticos para generar reportes administrativos y financieros.

- Cálculo del total de horas trabajadas semanalmente por cada chofer.
- Cálculo del sueldo total individual según el número de horas registradas y el valor por hora.
- Obtención del total general de pagos realizados por la empresa.
- Identificación del chofer que registró mayor cantidad de horas trabajadas el día lunes.

4.3. Controles Específicos

Dentro de la interfaz se implementaron controles específicos que permiten personalizar la información y mejorar la interacción del usuario.

- **CheckBox:** Utilizados para indicar bonos adicionales o el estado activo del chofer.
- **RadioButton:** Empleados para seleccionar el tipo de jornada laboral correspondiente.

- **TextFields:** Utilizados para ingresar nombres, horas de trabajo y valores salariales.
- **Buttons:** Implementados para generar cálculos y limpiar registros.

4.4. Código del programa

■ 4.5. Código en la carpeta controlador

Ruta: lib/controller/payrollcontroller.dart

Listing 8: Código del controlador

```
1 import '../models/driver_model.dart';
2
3 class PayrollController {
4   List<DriverModel> drivers = [];
5
6   void addDriver(DriverModel driver) {
7     if (drivers.length < 5) {
8       drivers.add(driver);
9     }
10  }
11
12  double calculateCompanyTotal() {
13    return drivers.fold(0, (sum, item) => sum + item.weeklySalary);
14  }
15
16  String getTopMondayDriver() {
17    if (drivers.isEmpty) return "N/A";
18    var topDriver = drivers.reduce(
19      (a, b) => a.dailyHours[0] > b.dailyHours[0] ? a : b,
20    );
21    return topDriver.name;
22  }
23 }
```

■ 4.6. Código en la carpeta model

Ruta: lib/model/drivermodel.dart

Listing 9: Código en la carpeta model

```
1 class DriverModel {
2   String name;
3   List<double> dailyHours; // 6 d as
4   double hourlyRate;
5   bool isActive;
6   String shift; // RadioButton: Diurna/Nocturna
7
8   DriverModel({
9     this.name = '',
10    List<double>? dailyHours,
11    this.hourlyRate = 0.0,
12    this.isActive = true,
13    this.shift = 'Diurna',
14  }) : dailyHours = dailyHours ?? List.filled(6, 0.0);
15
16  double get totalHours => dailyHours.reduce((a, b) => a + b);
17  double get weeklySalary => totalHours * hourlyRate;
18 }
```

■ 4.7. Código en la carpeta view

Ruta: lib/view/payrollformview.dart

Listing 10: Código en la carpeta view

```

1 import 'package:flutter/material.dart';
2 import '../controllers/payroll_controller.dart';
3 import '../models/driver_model.dart';
4 import '../widgets/atoms/custom_button.dart';
5 import '../widgets/molecules/labeled_input.dart';
6
7 class PayrollFormView extends StatefulWidget {
8   const PayrollFormView({super.key});
9
10  @override
11  State<PayrollFormView> createState() => _PayrollFormViewState();
12 }
13
14 class _PayrollFormViewState extends State<PayrollFormView> {
15   final _payrollCtrl = PayrollController();
16   final _nameCtrl = TextEditingController();
17   final _rateCtrl = TextEditingController();
18   final List<TextEditingController> _hourCtrls = List.generate(
19     6,
20     (_) => TextEditingController(),
21   );
22
23   DriverModel _currentDriver = DriverModel();
24
25   void _registerDriver() {
26     _currentDriver.name = _nameCtrl.text;
27     _currentDriver.hourlyRate = double.tryParse(_rateCtrl.text) ?? 0.0;
28     _currentDriver.dailyHours = _hourCtrls
29       .map((c) => double.tryParse(c.text) ?? 0.0)
30       .toList();
31
32     setState(() {
33       _payrollCtrl.addDriver(_currentDriver);
34       _clearInputs();
35     });
36
37     if (_payrollCtrl.drivers.length == 5) {
38       ScaffoldMessenger.of(context).showSnackBar(
39         const SnackBar(
40           content: Text(
41             "L mite de 5 choferes alcanzado. Puede ver el reporte.",
42           ),
43         ),
44       );
45     }
46   }
47
48   void _clearInputs() {
49     _nameCtrl.clear();
50     _rateCtrl.clear();
51     for (var c in _hourCtrls) {
52       c.clear();
53     }
54     _currentDriver = DriverModel();
55   }
56
57   @override
58   Widget build(BuildContext context) {
59     return Scaffold(
60       appBar: AppBar(
61         title: Text("Registro Choferes (${_payrollCtrl.drivers.length}/5)"),
62       ),
63       body: SingleChildScrollView(
64         padding: const EdgeInsets.all(16),
65         child: Column(
66           children: [
67             LabeledInput(label: "Nombre del Chofer", controller: _nameCtrl),
68             LabeledInput(
69               label: "Sueldo por Hora",
70               controller: _rateCtrl,
71               type: TextInputType.number,
72             ),
73             const Text(
74               "Horas por d a (Lun - Sab):",
75               style: TextStyle(fontWeight: FontWeight.bold),
76

```

```

77     ),
78     GridView.count(
79       shrinkWrap: true,
80       crossAxisCount: 3,
81       childAspectRatio: 2,
82       children: List.generate(
83         6,
84         (index) => Padding(
85           padding: const EdgeInsets.all(4.0),
86           child: TextField(
87             controller: _hourCtrls[index],
88             decoration: InputDecoration(labelText: "D a ${index + 1}"),
89             keyboardType: TextInputType.number,
90           ),
91         ),
92       ),
93     ),
94
95     RadioListTile(
96       title: const Text("Jornada Diurna"),
97       value: "Diurna",
98       groupValue: _currentDriver.shift,
99       onChanged: (v) => setState(() => _currentDriver.shift = v!),
100     ),
101     RadioListTile(
102       title: const Text("Jornada Nocturna"),
103       value: "Nocturna",
104       groupValue: _currentDriver.shift,
105       onChanged: (v) => setState(() => _currentDriver.shift = v!),
106     ),
107
108     CheckboxListTile(
109       title: const Text("Chofer Activo"),
110       value: _currentDriver.isActive,
111       onChanged: (v) => setState(() => _currentDriver.isActive = v!),
112     ),
113
114     const SizedBox(height: 20),
115     Row(
116       mainAxisAlignment: MainAxisAlignment.spaceEvenly,
117       children: [
118         CustomButton(
119           label: "Registrar",
120           onPressed: _registerDriver,
121           color: Colors.blue,
122         ),
123         CustomButton(
124           label: "Ver Reporte",
125           onPressed: () {
126             Navigator.pushNamed(
127               context,
128               '/report',
129               arguments: _payrollCtrl,
130             );
131           },
132           color: Colors.green,
133         ),
134       ],
135     ),
136   ],
137 ),
138 );
139 }
140 }
141 }

```

■ 4.8. Código en la carpeta widgets

• 4.9. Código en la carpeta atoms

Ruta: lib/wodgets/atoms/custombutton.dart

Listing 11: Código en la carpeta widgets/atoms

```

1 import 'package:flutter/material.dart';
2

```

```

3 class CustomButton extends StatelessWidget {
4   final String label;
5   final VoidCallback onPressed;
6   final Color? color;
7
8   const CustomButton({
9     super.key,
10    required this.label,
11    required this.onPressed,
12    this.color,
13  });
14
15  @override
16  Widget build(BuildContext context) {
17    return ElevatedButton(
18      style: ElevatedButton.styleFrom(
19        backgroundColor: color ?? Colors.blue,
20        foregroundColor: Colors.white,
21      ),
22      onPressed: onPressed,
23      child: Text(label),
24    );
25  }
26 }

```

Ruta: lib/wodgets/atoms/customtext.dart

Listing 12: Código en la carpeta widgets/atoms

```

1 import 'package:flutter/material.dart';
2
3 class CustomText extends StatelessWidget {
4   final String text;
5   final double fontSize;
6   final FontWeight fontWeight;
7   final Color color;
8
9   const CustomText({
10     super.key,
11     required this.text,
12     this.fontSize = 16.0,
13     this.fontWeight = FontWeight.normal,
14     this.color = Colors.black,
15   });
16
17   @override
18   Widget build(BuildContext context) {
19     return Text(
20       text,
21       style: TextStyle(
22         fontSize: fontSize,
23         fontWeight: fontWeight,
24         color: color,
25       ),
26     );
27   }
28 }

```

• 4.10. Código en la carpeta molecules

Ruta: lib/wodgets/atoms/labeledinput.dart

Listing 13: Código en la carpeta widgets/atoms

```

1 import 'package:flutter/material.dart';
2 import '../atoms/custom_text.dart';
3
4 class LabeledInput extends StatelessWidget {
5   final String label;
6   final TextEditingController controller;
7   final TextInputType type;
8
9   const LabeledInput({
10     super.key,
11     required this.label,
12     required this.controller,
13     this.type = TextInputType.text,
14   });

```

```
15 |
16 | @override
17 | Widget build(BuildContext context) {
18 |   return Column(
19 |     crossAxisAlignment: CrossAxisAlignment.start,
20 |     children: [
21 |       CustomText(text: label, fontWeight: FontWeight.bold),
22 |       TextField(controller: controller, keyboardType: type),
23 |       const SizedBox(height: 15),
24 |     ],
25 |   );
26 | }
27 | }
```

■ 4.11. Código en el archivo main.dart

Ruta: lib/main.dart

Listing 14: Código en la carpeta principal

```
1 | import 'package:flutter/material.dart';
2 | import 'views/payroll_form_view.dart';
3 | import 'views/payroll_report_view.dart';
4 |
5 | void main() => runApp(const PayrollApp());
6 |
7 | class PayrollApp extends StatelessWidget {
8 |   const PayrollApp({super.key});
9 |
10 |   @override
11 |   Widget build(BuildContext context) {
12 |     return MaterialApp(
13 |       debugShowCheckedModeBanner: false,
14 |       title: 'N mina Choferes',
15 |       theme: ThemeData(primarySwatch: Colors.blue, useMaterial3: true),
16 |       initialRoute: '/',
17 |       routes: {
18 |         '/': (context) => const PayrollFormView(),
19 |         '/report': (context) => const PayrollReportView(),
20 |       },
21 |     );
22 |   }
23 | }
```

4.12. Programa Funcional

En esta sección se presentan las capturas de pantalla correspondientes al funcionamiento de la aplicación desarrollada en Flutter. Las imágenes evidencian la implementación de la interfaz gráfica, la interacción con los controles básicos y la correcta ejecución de las funcionalidades programadas.

Se muestran los procesos principales del sistema, incluyendo el ingreso de datos, la selección de opciones mediante controles interactivos, el cálculo automático de resultados y la visualización de reportes generados por la aplicación.

Registro Choferes (1/5)

Nombre del Chofer

Mateo Condor

Sueldo por Hora

8.5

Horas por día (Lun - Sab):

Día 1	Día 2	Día 3
3	2	4
Día 4	Día 5	Día 6
3	5	1

- ☒ Jornada Diurna
- ☐ Jornada Nocturna

Chofer Activo



Registrar

Ver Reporte

← Reporte General

Total General Empresa: \$939.12

Más horas el Lunes: David Pilaguano

David Pilaguano

Horas: 37 | Sueldo: \$155.4



Mateo Condor

Horas: 18 | Sueldo: \$153



Alejandro Flores

Horas: 44 | Sueldo: \$126.72



Mateo Pinto

Horas: 30 | Sueldo: \$180



Emilio Paredes

Horas: 27 | Sueldo: \$324



5. Conclusiones

- **Eficacia de la Arquitectura MVC:** La implementación del patrón Modelo-Vista-Controlador permitió una separación clara de responsabilidades. Esto garantiza que la lógica de cálculo, como el procesamiento de consumos de servicios o la nómina de choferes, se mantenga aislada de la interfaz de usuario, facilitando el mantenimiento y la escalabilidad del código a futuro.
- **Modularidad con Atomic Design:** La organización de la carpeta de widgets bajo la metodología *Atomic Design* (*Átomos, Moléculas y Organismos*) fomentó la creación de componentes altamente reutilizables. Esta estructura no solo ordena el proyecto, sino que profesionaliza el desarrollo al estandarizar elementos visuales como botones y etiquetas de resultados.
- **Versatilidad en el Uso de Controles:** La integración de diversos controles básicos de Flutter, como *TextFields* para la entrada de datos, *CheckBox* para servicios adicionales y *RadioButtons* para la selección de categorías, demostró ser efectiva para capturar información compleja de manera intuitiva para el usuario.
- **Gestión Eficiente de la Navegación:** El uso de rutas en Flutter (*push*, *pop* y *pushNamed*) permitió establecer un flujo de usuario lógico y fluido entre las pantallas de registro y los resúmenes de pago o reportes generales, cumpliendo con el estándar de una aplicación móvil profesional.
- **Cumplimiento de Reglas de Negocio:** Se logró automatizar procesos financieros críticos, desde el cálculo de subtotales con tarifas específicas para servicios básicos, hasta la generación de estadísticas detalladas de horas trabajadas y sueldos semanales para una flota de transporte.

Repositorio GitHub: <https://github.com/DavidPilaguano/Moviles.git>